WK 3

BA LEARNING PLAN

Stakeholder Management and Delivery

Expectations, backlogs, Jira and running a sprint planning session

---

01 **Managing stakeholder expectations**
Analysis, engagement, trust, conflict and difficult conversations

02 **Managing the product backlog**
Structure, refinement, prioritisation, sizing and metrics

03 **Jira essentials**
Hierarchy, boards, workflows, JQL, reports and conventions

04 **Worked practice: mock backlog and sprint planning**
A full epic breakdown and a timed 15-minute facilitation script

Week 3 at a glance

Weeks 1 and 2 were about getting the analysis right. Week 3 is about the fact that being right is not sufficient - people have to agree, and work has to flow.

Course	The five things to take from it
Business Analysis: Managing Stakeholder Expectations	How expectations form and why they diverge; communication planning; building credibility; handling resistance and conflict; saying no without damaging the relationship
Agile Product Owner: Managing the Product Backlog	What a healthy backlog looks like; refinement as a habit; prioritisation frameworks; sizing and forecasting; the metrics that matter and the ones that mislead
Jira Essential Training	The issue hierarchy; Scrum versus Kanban boards; workflows and statuses; JQL; reports and dashboards

The deliverable

A mock backlog in a free Jira board for your week 1 feature: epics, broken into stories, prioritised, followed by a 15-minute mock sprint planning session that you record or present. Part 4 gives you a complete worked backlog and a minute-by-minute facilitation script.

EXAM / INTERVIEW TIP

The hardest part of the week 3 practice is not Jira - it is the fifteen minutes of talking. Write the script, rehearse it once, then record. Watching yourself facilitate is uncomfortable and it is the fastest feedback you will get all month.

Contents

Part 1: Managing Stakeholder Expectations 4

1.1 The Engagement Cycle . . . 4
1.2 Analysis Tools . . . 4
1.3 Where Expectation Gaps Come From . . . 5
1.4 Communication Planning . . . 6
1.5 Credibility and Trust . . . 7
1.6 Difficult Conversations . . . 7

Part 2: Managing the Product Backlog 9

2.1 The Product Owner . . . 9
2.2 Backlog Anatomy . . . 9
2.3 Refinement . . . 10
2.4 Prioritisation . . . 11
2.5 Roadmap, Release Plan and Backlog . . . 11
2.6 Sizing and Forecasting . . . 12
2.7 Metrics . . . 12
2.8 Backlog Anti-patterns . . . 13

Part 3: Jira Essentials 14

3.1 The Issue Hierarchy . . . 14
3.2 Anatomy of an Issue . . . 15
3.3 Boards, Backlogs and the Difference . . . 15
3.4 Running a Sprint in Jira . . . 16
3.5 JQL - Enough to Answer Your Own Questions . . . 17
3.6 The Reports Worth Reading . . . 17
3.7 Conventions That Keep a Board Usable . . . 18

Part 4: Worked Practice: Mock Backlog and Sprint Planning 19

4.1 Carrying the Feature Forward . . . 19

4.2 Writing the Epic 19
4.3 The Backlog 20
4.4 Prioritising - and Why the Methods Disagree 21
4.5 Setting It Up in Jira 22
4.6 The 15-Minute Sprint Planning Script 22
4.7 Assessing Your Own Recording 23

Appendix: Quick Reference and Self-Check **25**

PART 1

Managing Stakeholder Expectations

Why projects fail on relationships more often than on requirements

1

What this part covers

- The engagement cycle: identify, analyse, plan, engage, monitor
- Analysis tools beyond the power-interest grid
- Where expectation gaps come from and how to close them
- Building a communication plan that people actually read
- Establishing credibility and trust early
- Difficult conversations, saying no, and handling resistance
- Negotiation basics and escalation without damage

1.1 The Engagement Cycle

1. **Identify** - find everyone affected, including the ones nobody mentions.
2. **Analyse** - understand their influence, interest, attitude and what success looks like to them personally.
3. **Plan** - decide who gets what information, how often, in what form, and who owns each relationship.
4. **Engage** - do it, consistently, including when there is no good news.
5. **Monitor** - attitudes shift. Re-check after reorganisations, budget changes and any bad sprint.

KEY POINT

Stakeholder management is not a task you complete in week one. Attitudes change when circumstances change, and the stakeholder who was supportive during discovery may be the one blocking your release.

1.2 Analysis Tools

Power / interest grid

	Low interest	High interest
High power	**Keep satisfied** - consult on decisions that affect them, keep it brief, never surprise them	**Manage closely** - involve in decisions, co-create, seek explicit approval
Low power	**Monitor** - general communications, minimal effort	**Keep informed** - regular updates; often your best source of operational detail

The salience model - three dimensions instead of two

Attribute	Meaning	Implication
Power	Can they impose their will?	Ignore at your peril
Legitimacy	Is their claim appropriate and proper?	Deserves to be heard
Urgency	Does their claim demand immediate attention?	Drives sequencing

- All three present makes someone **definitive** - engage first, always.
- Power plus urgency without legitimacy makes someone **dangerous** - they can disrupt and will act fast, but their claim is not proper. This is the group most often mishandled.
- Legitimacy alone makes someone **discretionary** - worth engaging, unlikely to force the issue.

The stakeholder register - what to actually record

Field	Why it matters
Name, role, organisation	The basics
Interest in the outcome	What success looks like *to them*, not to the project
Influence and power	Formal authority and informal influence are different; record both
Current attitude	Champion, supportive, neutral, sceptical, resistant
Desired attitude	Where you need them to be, and the gap you must close
Preferred channel and cadence	Some people read email; some only respond in person
Who owns the relationship	One named person, or it becomes nobody's job
Notes on history	Past project experience shapes how they will react to yours

WATCH OUT

The two fields most registers omit are **current attitude** and **desired attitude**. Without them the register is a contact list. With them it is a plan: you can see at a glance which three relationships need work this month.

Other lenses

- **Onion diagram** - concentric rings by distance from the solution; good for showing who is directly affected versus contextually interested.
- **Influence network** - who listens to whom. Often the fastest route to a resistant executive is the person they trust, not a better slide deck.
- **RACI** - removes ambiguity about who decides. Exactly one Accountable per activity.

1.3 Where Expectation Gaps Come From

An expectation gap is the distance between what someone believes will happen and what will actually happen. Gaps do not usually come from lying; they come from silence and from imprecision.

Cause	What it sounds like	Prevention
Vague language	'We'll look at reporting in this phase'	State what is in and, explicitly, what is out
Optimism in early estimates	'Should be about a month'	Give ranges and state the assumptions behind them
Silence during bad news	Nothing for three weeks, then a slipped date	Report bad news early and at the same cadence as good news
Different definitions of done	'I thought it was finished'	Publish and refer to a Definition of Done
Unmanaged scope conversations	'You said you could add that'	Every request gets an impact assessment before an answer
Assuming shared context	'Obviously it has to integrate with billing'	Write assumptions down and get them confirmed

KEY POINT

The reliable formula: **satisfaction equals perception minus expectation.** You can improve satisfaction by delivering more, which is slow and expensive, or by managing expectations honestly from the start, which is neither. Most projects attempt only the first.

1.4 Communication Planning

Question	Decide
Who	Which stakeholder or group
What	The specific information they need - not everything you have
Why	What decision or action it enables. If none, do not send it
When	Frequency and timing, tied to their decision cycle rather than yours
How	Channel - email, stand-up, demo, one to one, dashboard
Who by	The named owner of that communication

- **Match the channel to the person**, not to your convenience. An executive who never reads email needs a five-minute conversation before the steering meeting, not a twelve-page status report.
- **Lead with the decision or the risk**, never with a chronology of what the team did.
- **Say the uncomfortable thing first.** Bad news buried in paragraph six reads as concealment when it surfaces later.
- **Repeat key messages.** People need to hear something several times before it registers, particularly if they did not want to hear it.

EXAMPLE

A status update that works: *'The reorder feature is on track for the 14th. One risk: the menu availability API returns restaurant-level data, not item-level. If that is not resolved by Friday we either ship without the substitution flow, or move two weeks. I need a decision from you by Thursday.'* Four sentences: status, risk, options, ask.

1.5 Credibility and Trust

- **Do the homework first.** Never make a stakeholder teach you what you could have read. It is the fastest way to lose an hour of goodwill.
- **Reflect their language back.** Using their terms correctly signals you understood; quietly correcting their terminology signals the opposite.
- **Close every loop.** If you said you would find out, come back even when the answer is 'still no news'.
- **Be visibly useful early.** One small thing fixed in week one buys more credibility than a perfect analysis in week six.
- **Admit what you do not know.** 'I don't know, I'll find out by Thursday' is stronger than a confident guess that turns out wrong.
- **Never surprise anyone in a public forum.** Bad news is delivered privately first.

1.6 Difficult Conversations

Saying no without damaging the relationship

1. **Acknowledge the need genuinely.** 'That would help your team - I can see why you want it.'
2. **Make the trade-off visible rather than refusing.** 'It's about two weeks. To fit it in the current release, something else moves out - which of these would you drop?'
3. **Give the decision to the person who owns it.** You are not refusing; you are presenting a choice to the person with the authority to make it.
4. **Offer a route.** 'I'll add it to the backlog with your rationale so it is ranked against everything else at the next refinement.'
5. **Record the outcome** so the same conversation does not repeat in a month.

KEY POINT

The BA rarely has authority to say no, and does not need it. Your job is to make the **cost of yes** visible so the right person can decide. That reframes you from obstacle to adviser.

Handling resistance

Underlying cause	How it presents	What helps
Fear of job change or loss	Endless objections about detail, nothing is ever ready	Name it directly and honestly; involve them in designing the new way of working

Underlying cause	How it presents	What helps
Not consulted early	'Nobody asked me' late in the project	Go back, genuinely listen, and visibly change something as a result
Past project scars	'We tried this in 2019 and it failed'	Ask what went wrong then; use it as risk input rather than dismissing it
Loss of status or control	Blocking, slow approvals, escalation	Find them a meaningful role in the change rather than routing around them
Genuine disagreement	Clear, specific, reasoned objections	Take it seriously - they may simply be right

WATCH OUT

Do not mistake the last row for the others. A stakeholder with a well-reasoned objection is an asset, and labelling them 'resistant' is how projects talk themselves into shipping something broken.

Negotiation basics

- **Separate positions from interests.** The position is 'I need it in the June release'; the interest may be 'I have a board commitment in July'. Interests usually have more than one solution.
- **Know your BATNA** - the best alternative if no agreement is reached. It tells you when to stop conceding.
- **Widen the pool before dividing it.** Phasing, a manual workaround for one release, or a reduced-scope version often satisfies both sides.
- **Attack the problem, not the person.** 'How do we get your reporting need met by July?' rather than 'your request is unreasonable'.

Escalation without damage

1. Try to resolve it at the level it arose. Escalating first burns relationships.
2. Tell the person you are escalating before you do it. Never let them find out from their own manager.
3. Escalate the **decision**, not the person: 'we need a call on X, here are two options and the impact of each'.
4. Bring a recommendation. An escalation with no recommendation is a complaint.
5. Record the decision and communicate it back to everyone involved.

PART 2

Managing the Product Backlog

Keeping an ordered, ready, honest list of what matters next

2

What this part covers

- Product owner accountabilities and the product goal
- Backlog anatomy and what DEEP means in practice
- Refinement as a habit rather than a meeting
- Prioritisation: MoSCoW, WSJF, Kano, cost of delay
- Roadmap, release plan and backlog - three different things
- Sizing, velocity and honest forecasting
- The metrics that help and the ones that corrupt behaviour
- Backlog anti-patterns

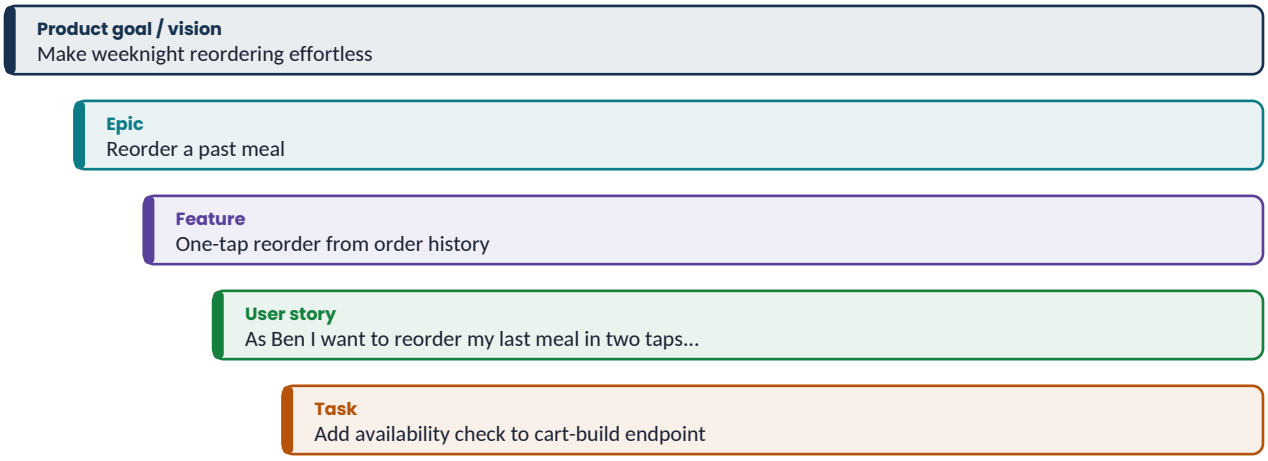
2.1 The Product Owner

Accountability	In practice
Maximising the value of the product	Saying no far more often than yes
Developing and communicating the product goal	One sentence everyone can repeat
Creating and ordering backlog items	A single ordered list, not priority buckets
Ensuring the backlog is transparent and understood	Anyone can see what is next and why
Accepting or rejecting work	The final call on whether a story is done

KEY POINT

The PO may delegate the work but keeps the accountability. In most organisations the BA does much of the analysis behind the backlog - splitting, criteria, modelling, stakeholder legwork - while the PO retains the decision on order.

2.2 Backlog Anatomy



From product goal to task - the backlog holds the middle three levels

DEEP - what healthy looks like

	Property	Test
D	Detailed appropriately	Top items ready to build; items three months out are one line
E	Estimated	Enough sizing to forecast, not precision on everything
E	Emergent	Items are added, changed and deleted as you learn
P	Prioritised	Strictly ordered, not grouped into High / Medium / Low

WATCH OUT

Priority **buckets** are a way of avoiding a decision. If forty items are all 'High', nobody has prioritised anything. Force a strict order - two items cannot both be next.

2.3 Refinement

Refinement is an ongoing activity, not a ceremony, typically consuming up to about ten percent of the team's capacity. Its purpose is that sprint planning is short and boring, because everything is already understood.

Activity	Who leads	Output
Split oversized items	BA / PO with the team	Stories that fit in a sprint
Write or sharpen acceptance criteria	BA with tester input	Testable boundaries
Clarify data, rules and dependencies	BA	Answers, or a spike where there are none
Estimate	The team	Relative sizes
Re-order	PO	An updated ordered backlog
Delete	PO	A smaller backlog - the most under-used move available

EXAM / INTERVIEW TIP

Arrive at refinement with the analysis already done - process model drawn, exception paths mapped, data checked, draft criteria written. That is what turns a two-hour argument into a twenty-minute decision, and it is the most visible value a BA adds to an agile team.

2.4 Prioritisation

Technique	Method	Best for
MoSCoW	Must, Should, Could, Won't - applied to a release, not the whole product	Timeboxed releases with a fixed date
Value versus effort	Two-by-two; take high value and low effort first	Fast triage during refinement
Cost of delay	What it costs the business per week of not having it	Making the price of waiting visible
WSJF	Cost of delay divided by job size; highest score goes first	Sequencing at scale, especially in SAFe
Kano	Basic expectations, performance attributes, delighters	Balancing table stakes against differentiation
Risk first	Tackle the riskiest assumption earliest	Novel technology or unproven business models
100-point method	Each stakeholder distributes 100 points	Groups with genuinely competing interests

WSJF worked through

Cost of delay is scored as the sum of user and business value, time criticality, and risk reduction or opportunity enablement. Each is scored relatively, often on a modified Fibonacci scale.

Item	Value	Time crit.	Risk / opp.	CoD	Size	WSJF	Order
One-tap reorder	8	5	3	16	5	3.2	1
Substitution flow	5	3	5	13	8	1.6	3
Price-change alerts	5	8	2	15	5	3.0	2
Reorder from web	3	2	1	6	8	0.75	4

EXAMPLE

Note what WSJF does here: price-change alerts score lower on raw value than the substitution flow, but they are time critical and much smaller, so they go second. That is the whole point - **small and urgent beats large and valuable** when you are sequencing, because finishing it frees capacity sooner.

2.5 Roadmap, Release Plan and Backlog

Artefact	Horizon	Granularity	Commitment level
Product roadmap	Quarters to a year	Themes and outcomes	Direction, not promise
Release plan	One to three months	Epics and features	Intent, revisited each sprint
Product backlog	Now to a few months	Stories at the top, epics below	Ordered, changes constantly
Sprint backlog	One sprint	Stories and tasks	The team's plan for the sprint goal

WATCH OUT

A roadmap of **features with dates** is a promise you cannot keep. A roadmap of **outcomes by quarter** - 'reduce time to reorder', 'increase repeat rate' - gives stakeholders the direction they want without committing the team to solutions nobody has validated.

2.6 Sizing and Forecasting

- **Story points** express relative size, combining volume, complexity and uncertainty. Not hours.
- **Modified Fibonacci** (1, 2, 3, 5, 8, 13, 20) is common because the widening gaps reflect growing uncertainty at larger sizes.
- **Reference story** - agree one well-understood item as a 3 and size everything against it.
- **Planning poker** - private estimates revealed simultaneously; the value is in the discussion between the highest and lowest estimator, not in the number.
- **Velocity** - the average completed per sprint over several sprints. A forecasting aid only.
- **Capacity** - time actually available next sprint after leave, support and holidays.

KEY POINT

Velocity is not a productivity metric. The moment it becomes a target, estimates inflate and the number stops describing reality. It is also meaningless across teams, because points are calibrated within a team.

2.7 Metrics

Metric	Tells you	How it gets abused
Velocity	Roughly how much fits in a sprint	Used as a target or to compare teams
Sprint burndown	Whether the sprint is tracking	Treated as a contract rather than a signal
Release burnup	Progress against total scope, and scope change	Ignored because it shows scope growth honestly
Cycle time	How long an item takes once started	Optimised by starting fewer things and calling it improvement
Cumulative flow	Where work piles up	Rarely abused, frequently ignored
Escaped defects	Quality reaching users	Suppressed by reclassifying bugs as stories

Metric	Tells you	How it gets abused
Outcome metrics	Whether the product actually helped	Not measured at all, which is the real problem

KEY POINT

Prefer a burn**up** to a burn**down** when reporting to stakeholders. A burndown hides scope growth - the line just stops falling. A burnup shows the total scope line rising, which is the conversation you actually need to have.

2.8 Backlog Anti-patterns

Anti-pattern	Why it hurts
The landfill	Thousands of items nobody deletes; real priorities become invisible
Everything is High priority	No decision has been made; the team picks for you
Stories as specifications	Huge items with no conversation; loses the shared understanding
Absent product owner	The team guesses at intent; rework follows
Horizontal slicing	Items organised by technical layer deliver nothing until the last one
Refinement skipped when busy	Sprint planning becomes a three-hour argument instead
No acceptance criteria until mid-sprint	Scope drifts inside the sprint; testing has no stable target
Technical debt invisible	It never competes for capacity, so it never gets done

PART 3

Jira Essentials

The tool most BA work is recorded in - hierarchy, boards, workflow, JQL and reports

3

What this part covers

- Issue hierarchy and issue types, and how they map to the backlog
- The fields on an issue that a BA actually owns
- Scrum boards, Kanban boards, and the difference between backlog and board
- Workflows, statuses and transitions
- Running a sprint: create, start, mid-sprint change, complete
- JQL - enough query language to answer your own questions
- The four reports worth reading and what each one tells you
- Naming and hygiene conventions that keep a board readable

3.1 The Issue Hierarchy

Everything in Jira is an 'issue' - a story, a bug, a task and an epic are all issues with different types. The hierarchy is what turns a flat list into a structure you can plan with.

Level	Default name	Meaning	Who typically writes it
Level 2	Initiative (premium plans only)	A strategic bucket spanning multiple epics and often multiple teams	Portfolio or programme lead
Level 1	Epic	A large body of work delivering one outcome; spans several sprints	Product owner, with BA support
Level 0	Story, Task, Bug, Spike	The unit the team commits to in a sprint; fits inside one sprint	BA or product owner
Sub-task	Sub-task	A slice of work inside a single story, owned by one person	The developer or tester doing the work

KEY POINT

An epic is **not** a folder for related work. It is an outcome with a beginning and an end. 'Reordering' is an epic because you can tell when it is finished. 'Mobile app' is not an epic - it never ends, and it will still be open in three years.

Standard issue types and when to use each

Type	Use it for	Needs acceptance criteria?
Story	A slice of user-facing value expressed from a user's perspective	Yes, always
Task	Necessary work with no direct user-facing behaviour, such as a config change	Yes - a definition of what 'done' means
Bug	Behaviour that differs from an agreed expectation	Steps to reproduce, actual result, expected result
Spike	Timeboxed investigation to answer a question	The question, the timebox and the required output
Sub-task	A unit of work within one story, usually created by the team	No - inherits the parent story's criteria

3.2 Anatomy of an Issue

Most fields on a Jira issue are filled in by someone else. These are the ones a business analyst owns and is judged on.

Field	What belongs in it	Common mistake
Summary	A short goal phrase readable on a board card, ideally verb-first	Writing a whole sentence, or a vague label like 'reorder work'
Description	The user story, context, links to the model or design, and any assumptions	Pasting a full specification nobody reads
Acceptance criteria	Binary, testable conditions - often a custom field or the bottom of the description	Leaving it empty and 'explaining it in stand-up'
Story points	Relative size, set by the team	The BA or manager filling this in on the team's behalf
Priority	The business ordering signal	Everything set to High, which conveys nothing
Epic link / parent	The outcome this contributes to	Orphan stories that never roll up to anything
Labels	Lightweight cross-cutting tags such as needs-design or regulatory	Dozens of near-duplicate labels created ad hoc
Components	The part of the product the work touches	Used as a second priority field
Fix version	The release the item is targeted at	Never set, so release reporting is impossible
Linked issues	Blocks, is blocked by, relates to, duplicates	Dependencies discussed verbally and never recorded
Attachments	Wireframes, BPMN model, sample data, the traceability reference	Screenshots with no explanation of what they show

EXAM / INTERVIEW TIP

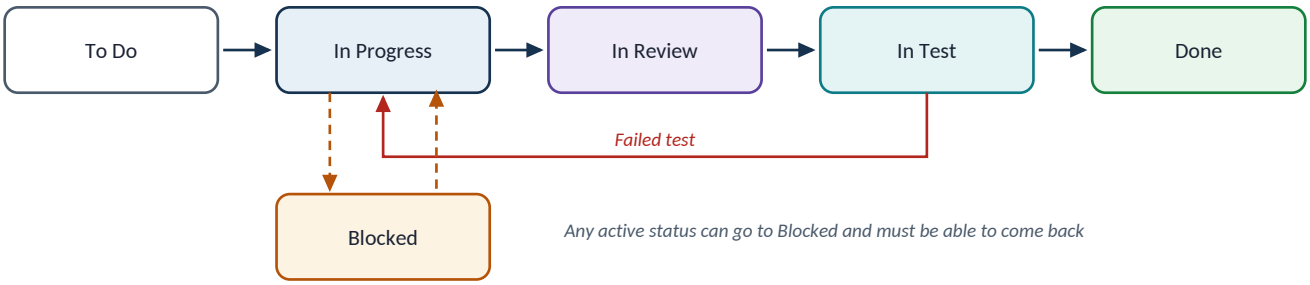
Write the **Summary** as if the reader can see nothing else - because on a board, they cannot. 'Reorder a past order in two taps' tells you what it is. 'Reorder - phase 1' tells you nothing at all.

3.3 Boards, Backlogs and the Difference

This trips up almost everyone new to Jira. The **backlog** is the ordered list of everything not yet in a sprint. The **board** shows only the work in the current sprint, arranged by status. They are two views of the same project.

	Scrum board	Kanban board
Work is batched into	Fixed-length sprints	A continuous flow
Has a separate backlog screen	Yes	Optional backlog, plus the board
Commitment	A sprint goal and a selected set of items	Whatever is pulled next
Key constraint	The sprint timebox	Work-in-progress (WIP) limits per column
Main metric	Velocity and sprint burndown	Cycle time and cumulative flow
Best when	Work is plannable and the team wants a rhythm	Work arrives unpredictably, such as support or operations

A typical workflow



Statuses become board columns; transitions define what a person is allowed to do next

WATCH OUT

Adding a status is not free. Every extra column is a queue where work sits still. If a column is always full, it is not a status - it is a bottleneck with a name. Three to five columns suits most teams.

3.4 Running a Sprint in Jira

1. **Refine first.** Items at the top of the backlog have criteria, an estimate and no open questions before planning starts.
2. **Create the sprint** on the Backlog screen and give it a name that states the goal, not just 'Sprint 14'.
3. **Drag items in** until the total points approach the team's average velocity, adjusted for leave.
4. **Start the sprint**, setting the start and end dates and writing the sprint goal in the field provided. The goal is what you protect when things go wrong.

5. **During the sprint**, the team moves cards across the board. If new work must come in, something of equal size comes out - and that trade is a conversation with the product owner, not a silent drag.
6. **Complete the sprint.** Jira asks what to do with unfinished items; they return to the backlog or move to the next sprint. Do not simply extend the sprint - that destroys velocity data.

KEY POINT

The **sprint goal** is the single most under-used field in Jira. It is what lets a team say 'we can drop that story and still succeed'. Without it, every item looks equally mandatory and the sprint fails as a whole or not at all.

3.5 JQL - Enough to Answer Your Own Questions

Jira Query Language is the difference between asking someone for a report and getting the answer in ten seconds. The structure is field, operator, value, joined by AND / OR and optionally sorted.

Question you actually have	JQL
What is in the current sprint for my project?	project = FD AND sprint in openSprints()
What did I create that still has no acceptance criteria?	project = FD AND reporter = currentUser() AND description !~ "Given"
What is blocked right now?	project = FD AND status = Blocked ORDER BY updated ASC
Everything under my epic, in order	project = FD AND parent = FD-101 ORDER BY rank ASC
What changed since I last looked?	project = FD AND updated >= -3d ORDER BY updated DESC
Unestimated stories near the top of the backlog	project = FD AND type = Story AND "Story Points" is EMPTY AND sprint is EMPTY
Bugs raised against the reorder feature this release	project = FD AND type = Bug AND component = Reorder AND fixVersion = 2.4
Anything untouched for a fortnight	project = FD AND status not in (Done, Closed) AND updated <= -14d

Operators worth memorising

- **=** and **!=** for exact matches; **in (...)** and **not in (...)** for lists.
- **~** means 'contains text' and **!~** means 'does not contain' - useful for searching descriptions.
- **is EMPTY** and **is not EMPTY** for finding missing fields; this is how you audit your own backlog hygiene.
- **Relative dates**: -3d, -2w, -1M. Combine with **updated**, **created** or **resolved**.
- **Functions**: currentUser(), openSprints(), closedSprints(), startOfWeek(), membersOf().
- **ORDER BY rank** gives you backlog order; ORDER BY priority does not.

EXAM / INTERVIEW TIP

Save the 'unestimated stories near the top' and 'no acceptance criteria' queries as filters and check them before every refinement session. That single habit makes you visibly more organised than most people on the team.

3.6 The Reports Worth Reading

Report	What it shows	What to look for
Sprint burndown	Remaining work against days left in the sprint	A flat line for several days means work is not being broken down or is stuck in review
Velocity chart	Committed versus completed points per sprint	Stability matters more than the number. Wild swings mean estimation or scope problems
Cumulative flow diagram	The number of items in each status over time	A widening band is a bottleneck; a growing 'In Review' band is the classic one
Control chart	Cycle time per completed issue	Outliers are the interesting part - find out what made those items different
Epic burndown / report	Progress toward completing an epic	Scope added to the epic mid-flight, which is where release dates quietly slip

WATCH OUT

Do not present velocity to stakeholders as a productivity figure. Report **outcomes delivered** and use velocity only to forecast. The moment velocity becomes a target, the estimates inflate and you lose your only forecasting tool.

3.7 Conventions That Keep a Board Usable

- **One summary style.** Verb-first goal phrases, sentence case, no ticket prefixes duplicated in the title.
- **Acceptance criteria in the same place every time** - a custom field or a fixed heading in the description. Never scattered through comments.
- **Link, do not describe, dependencies.** 'Blocked by FD-118' is checkable; 'waiting on the payments team' is not.
- **Attach the model.** The BPMN diagram, the wireframe and the data question all belong on the story, so the developer never has to hunt.
- **Close the loop in the ticket.** Decisions made in a meeting go into the ticket the same day, or they did not happen.
- **Delete ruthlessly.** Anything untouched for six months is noise. If it matters it will come back.
- **Do not use Jira as the requirements document** for anything with regulatory weight. Link out to the specification, keep the ticket as the delivery record.

KEY POINT

The test of a good board: a developer who was on leave for a week should be able to pick up any card at the top of the backlog and start work without messaging anyone. If they cannot, the analysis is not finished - regardless of how full the board looks.

PART 4

Worked Practice: Mock Backlog and Sprint Planning

4

The week 1 feature, taken all the way to a board and a facilitated session

What this part covers

- Turning the week 1 BRD into an epic with a written outcome and success measure
- A complete backlog: eleven items, sized, ordered and justified
- Prioritisation applied three ways, and why they disagree
- Setting the board up in Jira, field by field
- A minute-by-minute script for a 15-minute sprint planning session
- How to assess your own recording afterwards

4.1 Carrying the Feature Forward

This is the same 'reorder past meal' feature from week 1 and the same process model from week 2. Nothing new is invented here - the point of the exercise is that artefacts should connect.

Week	Artefact you produced	What it becomes this week
Week 1	One-page BRD with objectives OBJ-1 to OBJ-3 and requirements BR-01 to BR-05	The epic description and its success measures
Week 1	Five user stories with acceptance criteria	Five of the backlog items, largely unchanged
Week 2	BPMN model with the availability gateway and substitution path	The stories nobody thinks of until the model exposes them
Week 2	SQL queries on order history	The evidence that justifies the epic and sets the baseline measure

KEY POINT

When your mentor reviews this, the thing being assessed is not the board. It is whether the epic traces back to a business objective and whether every story traces to something in the process model. A tidy board with no traceability is the most common way this exercise is failed.

4.2 Writing the Epic

Epic FD-100 - One-tap reorder

Field	Content
Summary	One-tap reorder of a past meal
Outcome	Repeat customers can re-place a previous order in under 45 seconds, with any changes to availability or price made visible before they pay
Why now	Order-history analysis shows a large share of orders are repeats of a prior order from the same restaurant, currently rebuilt by hand every time
Success measures	OBJ-1 repeat-order share up from the measured baseline; OBJ-2 median time from app open to checkout below 45 seconds; OBJ-3 basket abandonment on repeat orders below 12 percent
In scope	Reordering a complete past order from history in the mobile apps, revalidation of items, price and opening hours, amending the basket before payment, orders from the last 90 days
Out of scope	Scheduled or recurring orders, reordering from a different branch, group orders and split payment, the web application
Done when	All Must-have stories are accepted, the measures above are instrumented and reporting, and the feature is enabled for 100 percent of users on both platforms

EXAM / INTERVIEW TIP

Write the **Done when** line before you write a single story. It stops the epic becoming an open-ended folder, and it is the sentence you will use in the sprint review to claim the epic is finished.

4.3 The Backlog

Eleven items under the epic. Five come straight from the week 1 stories; four come from questions the week 2 process model raised; two are the enabling and measurement work that gets forgotten until it is too late.

Ref	Item	Type	Pts	MoSCoW	Where it came from
FD-101	See my past orders with what I paid	Story	3	Must	US-01, BRD BR-01
FD-102	Reorder a past order in one tap	Story	5	Must	US-02, BRD BR-01
FD-103	Revalidate items, price and opening hours before checkout	Story	8	Must	BPMN: the revalidation task
FD-104	Show me what is unavailable and offer substitutes	Story	8	Must	BPMN: the availability gateway
FD-105	Tell me if the price changed before I pay	Story	3	Must	US-04, BRD BR-02
FD-106	Edit the rebuilt basket before paying	Story	5	Should	US-05
FD-107	Handle the restaurant being closed right now	Story	3	Should	BPMN: opening-hours check
FD-108	Handle the restaurant declining after payment	Story	5	Should	BPMN: exception path
FD-109	Instrument the three success measures	Task	3	Must	Epic success measures
FD-110	Spike: can the pricing service quote a historic basket?	Spike	2	Must	Unknown found in refinement

Ref	Item	Type	Pts	MoSCoW	Where it came from
FD-111	Reorder entry point on the home screen	Story	3	Could	Design suggestion

WATCH OUT

Notice FD-110 is a **spike sized at 2 points and marked Must**. FD-103 cannot be estimated honestly until that question is answered. Putting the spike in the sprint *before* the story it unblocks is the single most useful sequencing decision in this backlog.

Two stories written out in full

FD-104	Show me what is unavailable and offer substitutes
Story	As Ben, a weeknight regular, I want to be told which items from my past order are unavailable and what I could have instead, so that I do not discover the problem after I have paid
Acceptance criteria	Given my past order contains an item that is out of stock, when I tap reorder, then that item is shown flagged as unavailable before the payment screen. Given an unavailable item has a substitute defined by the restaurant, when the gaps are shown, then the substitute is offered with its price difference stated. Given no substitute is defined, when the gaps are shown, then I can remove the item or cancel the reorder. Given every item is available, when I tap reorder, then no substitution screen is shown at all.
Notes	Links to the BPMN model, exclusive gateway 'All items available?'. Blocked by nothing; blocks FD-106.

FD-110	Spike: can the pricing service quote a historic basket?
Question	Given a basket composed from an order up to 90 days old, can the pricing service return a current price for every line, including items whose modifiers have since changed?
Timebox	One day, one developer
Output	A written answer in the ticket, a recommendation, and a re-estimate of FD-103. If the answer is no, a note on what the fallback costs.

4.4 Prioritising - and Why the Methods Disagree

Run more than one method and compare. Where they agree you can move quickly; where they disagree you have found the decision that actually needs a human.

Item	MoSCoW	Value vs effort	WSJF-ish view	Where it lands
FD-101 See past orders	Must	High value, low effort	High - unblocks everything	Sprint 1, first
FD-102 One-tap reorder	Must	High value, medium effort	Highest - it is the feature	Sprint 1

Item	MoSCoW	Value vs effort	WSJF-ish view	Where it lands
FD-110 Pricing spike	Must	No direct value	High - removes risk early	Sprint 1, before FD-103
FD-103 Revalidation	Must	High value, high effort	High - legal and trust risk	Sprint 1 if spike allows
FD-104 Substitutes	Must	High value, high effort	Medium - can follow	Sprint 2
FD-109 Instrumentation	Must	No user value, low effort	High - without it you cannot prove OBJ-1	Sprint 1, small
FD-111 Home screen entry	Could	Medium value, low effort	Low	Deferred - reassess after launch

KEY POINT

FD-109 is the item that exposes whether you have understood the point. It delivers **no user value at all**, so value-versus-effort ranks it low - and yet without it you cannot demonstrate a single one of the epic's success measures. Be ready to defend it; mentors ask about exactly this.

4.5 Setting It Up in Jira

1. Create a free Jira account and a **team-managed Scrum project**. Team-managed is simpler and you can configure it yourself without an admin.
2. Set the project key to something short - FD works for this example - so issue keys read cleanly.
3. Create the epic first, on the backlog screen, using the Epic panel. Paste in the outcome, measures and 'Done when' from section 4.2.
4. Create the eleven items and set each one's **parent** to the epic, so they roll up.
5. Add acceptance criteria in the same place on every ticket - either a custom field or a bold 'Acceptance criteria' heading at the foot of the description.
6. Order the backlog by dragging. Rank order is the real priority; the Priority field is only a label.
7. Estimate in story points. If you are working alone, size relative to FD-101 as your reference 3.
8. Link FD-110 to FD-103 with 'blocks', so the dependency is visible rather than remembered.
9. Create Sprint 1, drag in roughly the capacity you decided, and write a real sprint goal.

EXAM / INTERVIEW TIP

Give yourself a plausible velocity before you plan, or the exercise has no constraint and every item goes in. Assume a small team with a velocity of about 20 points. That forces the trade-off the exercise is designed to teach.

4.6 The 15-Minute Sprint Planning Script

A real sprint planning for a two-week sprint runs to a couple of hours. Fifteen minutes means you are demonstrating the *shape* of the session, so be explicit about what you are compressing. Say so at the start - it reads as competence, not as an excuse.

Time	Segment	What you actually say and do
0:00 - 1:30	Frame the session	State the sprint length, the team's assumed velocity of 20 points, and who is in the room. Say plainly: 'in a real session the team estimates live and this runs two hours; today I am walking the structure and the decisions.'
1:30 - 3:00	Propose the sprint goal	'A customer can find a past order and reorder it, and we know whether it is faster than ordering from scratch.' Then explain why that goal, not a list of tickets - it is what lets you drop scope later without failing.
3:00 - 6:00	Walk the candidate items	Take FD-101, FD-102, FD-110, FD-109 in order. For each: one sentence of purpose, the acceptance criteria headline, the estimate, and any dependency. Do not read the tickets aloud - summarise them.
6:00 - 9:00	Handle the risk item	Explain FD-110 the spike: why FD-103 cannot be sized until it is answered, why it goes first, and what happens to the sprint if the answer is no. This is the segment that shows analytical thinking.
9:00 - 11:30	Do the capacity maths out loud	3 + 5 + 2 + 3 = 13 points committed, leaving roughly 7. Ask whether FD-103 at 8 points fits. Conclude that it does not fit honestly, and either take FD-105 at 3 or leave slack for the spike's outcome. Make the trade-off visible.
11:30 - 13:30	Confirm the commitment and the risks	Read back the selected items, the goal, the one dependency and the one assumption (that the pricing service can quote historic baskets). Name who owns the spike.
13:30 - 15:00	Close	State what happens next: refinement mid-sprint for FD-104 and FD-106, the review date, and the one decision you need from the product owner before Thursday.

Phrases that make you sound like you have done this before

- 'The goal is X - so if we have to drop something, we drop the item that is not serving X.'
- 'That is an assumption rather than a fact, so I have written it on the ticket and we will know by Wednesday.'
- 'I cannot size that honestly yet, so I would rather spend a day finding out than guess at eight points.'
- 'That is new scope. Happy to take it - what comes out?'
- 'Can a tester write a test from that criterion without asking me a question?'

WATCH OUT

The most common way this recording goes wrong is reading the tickets aloud one by one. That is a status update, not planning. Planning is **selecting** work against a goal and a capacity, and being seen to make trade-offs.

4.7 Assessing Your Own Recording

Check	Weak	Strong
Sprint goal	A list of ticket numbers	One sentence of outcome that survives dropping a story
Traceability	Stories exist in isolation	Every story names the BRD requirement or the model element it came from
Capacity	Everything selected fits by coincidence	Arithmetic done out loud, and something consciously left out

Check	Weak	Strong
Risk	Unknowns not mentioned	The spike is sequenced first and its failure case is described
Criteria	'It should work properly'	Binary conditions a tester could execute unaided
Non-functional	Not mentioned at all	At least one - the 45-second target - is stated and located
Facilitation	You talk for fifteen minutes	You ask questions, pause, and record decisions and owners

Appendix — Quick Reference and Self-Check

The week 3 material compressed, then questions to test yourself against.

A.1 Stakeholder and Delivery Quick Reference

Concept	The compressed version
Power / interest grid	High power high interest: manage closely. High power low interest: keep satisfied. Low power high interest: keep informed. Low power low interest: monitor
RACI	Exactly one Accountable per row. No row without a Responsible. Watch for the person who is A on everything
Engagement levels	Unaware, resistant, neutral, supportive, leading - plot current and desired, and plan the gap
Expectation gap sources	Unstated assumptions, silence, optimism in status reporting, scope agreed with the wrong person
DEEP backlog	Detailed appropriately, Estimated, Emergent, Prioritised
Refinement	Ongoing, roughly 5-10 percent of capacity; split, clarify, estimate, re-order, remove
WSJF	Cost of delay divided by job size; highest first
Velocity	Forecasting aid only. Never a target, never compared between teams
Sprint goal	The sentence that lets you drop a story and still succeed
Jira hierarchy	Initiative, epic, story or task or bug or spike, sub-task
Backlog vs board	Backlog is everything not yet in a sprint; the board is the current sprint arranged by status
JQL essentials	field operator value, joined by AND / OR; is EMPTY finds gaps; ORDER BY rank gives true backlog order

A.2 Self-Check Questions

Stakeholder management

A senior stakeholder was not consulted and is now blocking the release. What went wrong and what do you do?

The stakeholder analysis missed them, or classified their influence too low. Do not argue the past: meet them, find out what they actually need, assess the impact honestly, and take the trade-off to the decision maker. Then update the register so it does not recur.

How do you handle a stakeholder who agrees in meetings and objects afterwards?

Agreement in the room is not agreement. Write decisions down and circulate them the same day, ask directly for objections rather than for agreement, and speak to them one-to-one beforehand - some people will not disagree in front of their peers.

Name the five engagement levels.

Unaware, resistant, neutral, supportive, leading. Plot where each stakeholder is now and where they need to be; the gap is your engagement plan.

What do you do when two stakeholders give you contradictory requirements?

Do not pick one. Make the conflict explicit, document both positions and their reasons, identify the shared objective above them, and take it to whoever is accountable for that objective. Escalating a decision is not failure; hiding a conflict is.

How do you deliver bad news about a slipping date?

Early, with the reason, the options, and a recommendation. Never let the first signal be a missed date. A stakeholder can absorb almost any news except a surprise.

Backlog and product ownership

Who owns the product backlog, and who owns the estimates?

The product owner owns the content and the order of the backlog. The team owns the estimates. Neither can override the other on their own side of that line.

What is wrong with a backlog where every item is priority High?

No decision has been made. Priority is only meaningful as an order. If everything is High, the team will pick for you - and they will pick on convenience, not value.

Why is a burnup usually better than a burndown for stakeholders?

A burndown hides scope growth - the line simply stops falling. A burnup shows the total scope line rising separately from work completed, which is the conversation that needs to happen.

When should technical debt appear in the backlog?

Always, sized and ordered like any other item. If it never competes for capacity it never gets done, and the cost surfaces later as unexplained slowdown.

Jira and the practice

Difference between the backlog and the board?

The backlog is the ordered list of everything not yet in a sprint. The board shows only the current sprint's work, arranged by status. Same project, two views.

Write JQL to find stories in your project with no story points that are not yet in a sprint.

project = FD AND type = Story AND "Story Points" is EMPTY AND sprint is EMPTY

Why should a spike be sequenced before the story it relates to?

Because the story cannot be estimated or designed honestly until the unknown is resolved. Sequencing the spike first converts an unmanaged risk into a one-day, budgeted question.

Your sprint ends with two stories unfinished. What do you do in Jira, and what do you not do?

Complete the sprint and let the unfinished items return to the backlog or move to the next sprint. Do not extend the sprint - it destroys the velocity data that is your only forecasting tool, and it hides the fact that the commitment was too large.

In your mock planning session, why include an item with no user value?

Instrumentation. Without it none of the epic's success measures can be evidenced, so you could deliver the whole feature and be unable to say whether it worked.

EXAM / INTERVIEW TIP

Three weeks of material reduces to one habit: **every artefact should point at another one**. The objective points at the requirement, the requirement at the process step, the process step at the story, the story at the acceptance criterion, the criterion at the test. When a mentor or an interviewer probes, they are almost always checking whether those links exist.